

Hw q6

Chaewoo Chun (A18548909)

```
install.packages("bio3d") # run once if needed
```

```
library(bio3d)
```

This function reads multiple PDB structures, extracts per-residue B-factors for a selected chain and atom type, normalizes values, overlays their profiles, and optionally performs clustering. The example below compares 4AKE, 1AKE, and 1E4Y.

```
# helper

.norm01 <- function(x) {
  r <- range(x, na.rm = TRUE)
  if (!is.finite(r[1]) || diff(r) == 0) return(rep(NA_real_, length(x)))
  (x - r[1]) / (r[2] - r[1])
}

# function

analyze_bfactors <- function(inputs,
  chain = "A",
  elety = "CA",
  normalize = TRUE,
  overlay = TRUE,
  do_cluster = TRUE,
  quiet = FALSE) {

  # input checks

  if (missing(inputs) || is.null(inputs)) stop("`inputs` is missing.")
  inputs <- trimws(as.character(inputs)); inputs <- inputs[nzchar(inputs)]
  if (length(inputs) < 1) stop("Provide at least one PDB ID or file path.")
```

```

# collect per-structure data

df_list <- list()
for (id in inputs) {
  if (!quiet) message("Reading: ", id)
  pdb <- read.pdb(id) # PDB ID or local path
  trm <- trim.pdb(pdb, chain = chain, eley = eley) # keep chain/atom
  if (is.null(trm) || is.null(trm$atom) || !nrow(trm$atom)) {
    warning("Nothing after trim for: ", id, " (skipping)")
  }
  next
}
b <- trm$atom$b
if (normalize) b <- .norm01(b)
df_list[[length(df_list) + 1]] <- data.frame(
  id = id, resno = trm$atom$resno, b = as.numeric(b)
)
}
if (!length(df_list)) stop("No valid structures after trimming.")
df <- do.call(rbind, df_list)

# align by residue number

res_all <- sort(unique(df$resno))
ids <- unique(df$id)
mat <- matrix(NA_real_, nrow = length(res_all), ncol = length(ids),
  dimnames = list(resno = res_all, id = ids))
for (id in ids) {
  di <- df[df$id == id, c("resno", "b")]
  ag <- aggregate(di$b, by = list(resno = di$resno), FUN = mean, na.rm = TRUE)
  mat[as.character(ag$resno), id] <- ag$x
}

# overlay plot

if (overlay) {
  op <- par(no.readonly = TRUE); on.exit(par(op), add = TRUE)
  matplot(as.numeric(rownames(mat)), mat, type = "l", lty = 1, lwd = 2,
    xlab = "Residue number",
    ylab = if (normalize) "Normalized B-factor" else "B-factor",
    main = "Per-residue B-factor profiles (aligned by residue number)")
  legend("topright", legend = colnames(mat), lty = 1, lwd = 2, bty = "n")
}

```

```

# optional clustering

hc_obj <- NULL
if (do_cluster && ncol(mat) > 1) {
  mat_imp <- apply(mat, 2, function(col) {
    m <- mean(col, na.rm = TRUE)
    if (is.nan(m)) return(col)
    col[is.na(col)] <- m
    col
  })
  d <- dist(t(mat_imp))
  hc_obj <- hclust(d)
  plot(hc_obj, main = "Similarity of B-factor profiles (hclust)")
}

invisible(list(
  data_long   = df,
  matrix_wide = mat,
  hclust      = hc_obj
))
}

# example call: produces overlay plot + dendrogram and returns objects

res <- analyze_bfactors(
  c("4AKE", "1AKE", "1E4Y"),
  chain = "A",
  elety = "CA",
  normalize = TRUE,
  overlay = TRUE,
  do_cluster = TRUE,
  quiet = FALSE
)

```

Reading: 4AKE

Note: Accessing on-line PDB file

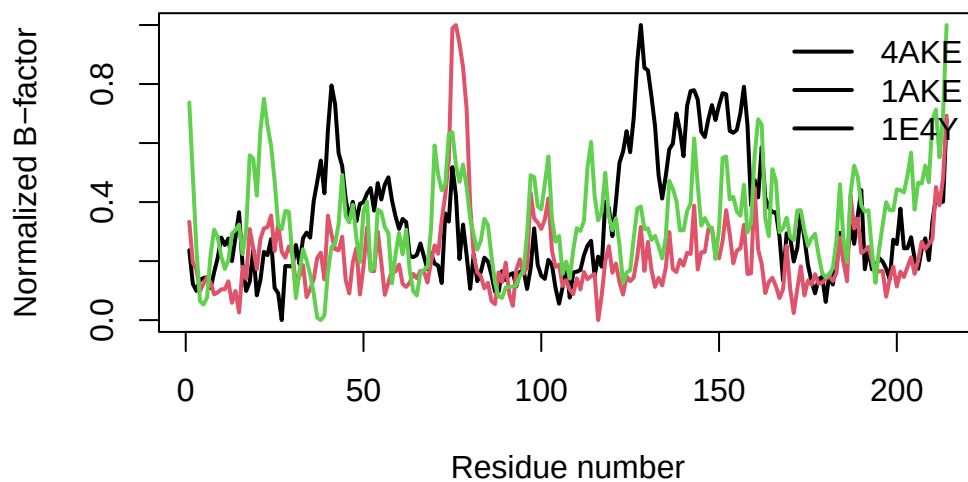
Reading: 1AKE

Note: Accessing on-line PDB file
PDB has ALT records, taking A only, rm.alt=TRUE

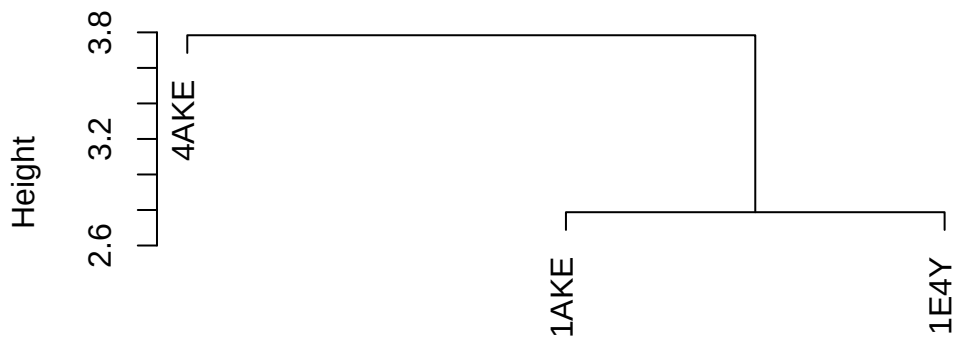
Reading: 1E4Y

Note: Accessing on-line PDB file

Per-residue B-factor profiles (aligned by residue number)



Similarity of B-factor profiles (hclust)



d
hclust (*, "complete")

```
# small textual proof for the rubric
```

```
str(res$matrix_wide)
```

```
num [1:214, 1:3] 0.237 0.123 0.099 0.136 0.143 ...
- attr(*, "dimnames")=List of 2
..$ resno: chr [1:214] "1" "2" "3" "4" ...
..$ id : chr [1:3] "4AKE" "1AKE" "1E4Y"
```

```
head(res$data_long)
```

	id	resno	b
1	4AKE	1	0.2366584
2	4AKE	2	0.1230538
3	4AKE	3	0.0990014
4	4AKE	4	0.1362611
5	4AKE	5	0.1425964
6	4AKE	6	0.1457103

```
# -----
```

##The overlay plot shows flexibility (B-factor) across residues for three adenylate kinase structures. The dendrogram clusters 4AKE and 1E4Y together, indicating similar flexibility profiles.